Probabilistic Performance Guarantees for Unseen Service Compositions and Configurations

Boris Sedlak, *Member, IEEE*, Milos Gravara, *Member, IEEE*, Víctor Casamayor Pujol, *Member, IEEE*, Stefan Nastic, *Member, IEEE*, Schahram Dustdar, *Fellow, IEEE*

***Abstract*—Note: TODOs and red texts indicate parts that are still work in progress**

***Index Terms*—Performance Analysis, Service Level Objectives, Gaussian Processes, Service Composition, Evolving Services**

I. Introduction

TODO: Move parts of the text to the background section.

Large Language Models (LLMs) have risen as universal interpretation and generation engines for text-based contents, introducing a major disruption for software development [7]. Prior to that, computing services were developed manually according to stakeholder requirements, and operated in close-loop controls to continuously meet operational thresholds derived from these very requirements. Within that context, LLMs appeared as a fast and cost-efficient extension to enable autonomous software development and operation, enabling a new branch of service research [1]: automatic composition [17] and generation [6], [13] of software services. The combined vision from these works is clear: a *future internet*, driven by high-level client requirements that determine how agents generate and compose services into software applications.

Traditional service-oriented architectures already provide clear roles and interfaces to operate such systems [18]: the service provider and the service consumer establish Service Level Agreements (SLAs) that specify service-quality commitments through measurable Service Level Objectives (SLOs). In the future internet, meeting these requirements involves dynamically creating, composing, and reconfiguring services, resulting in a large space of possible service compositions and configurations. Yet, evaluating each candidate through end-to-end benchmarking is costly and infeasible for large solution spaces. Service providers therefore require models that can estimate the performance and value of service configurations and compositions from limited empirical evidence, while accounting for the risk of SLA violations.

TODO: Not clear what (re-)configuration actually means.

For a service composition S with a topology $\{s_1 \rightarrow s_2 \rightarrow \dots\}$ such a predictive model should support service providers in answering questions like:

- What performance can we guarantee for the service composition as it is?
- If we reconfigure service s_1, how does this affect end-to-end performance?"
- If we replace s_1 with an arbitrary s_x, how does this affect the performance of the composition?

These questions arise in two complementary forms of service adaptation:

Service composition enables providers to create and compose services according to client SLOs. However, newly created or recomposed pipelines may have little or no end-to-end empirical evidence: a particular implementation may only be relevant to a single client, while pipelines may be recomposed dynamically in response to runtime conditions.

Service configuration enables operators to adjust the functionality and resource allocation of individual services. Thus, one service implementation can be used to satisfy the requirements of different clients over time.

Service composition and multi-dimensional configuration both create a vast search space, making it increasingly difficult to solve both problems simultaneously. From an organizational standpoint, it might be highly beneficial to clearly separate these problems: imagine an architecture where multiple competing providers aim to sell their services by providing optimistic SLAs, and one compositor—itself a service—that assembles applications according to client requirements. However, from a logical point, service configuration and service composition are highly entangled: with limited knowledge about the performance of individual services, it becomes even more difficult to identify good compositions.

Still, adjusting the two processes—the composition and the configuration—is decoupled: we can reconfigure services without changing their service pipeline, and vice-versa. Note, that the complexity for both problems can be reduced, either by bounding the number of possible compositions, or the number of service configurations. For service providers, this presents a tradeoff: should they offer more configurations to enable more flexible composition and operation, but at the cost of having less information about the feasibility of their own offer?

Although optimization techniques such as branch-and-bound can reduce this search space, the fundamental challenge remains the efficient evaluation of candidate solutions. Any learning-based optimizer must compare configurations for which different amounts of empirical evidence are avail-

B. Sedlak, P. Raith, A. Morichetta, and S. Dustdar are with the Distributed Systems Group, TU Wien, Vienna, Austria (e-mail: b.sedlak@dsg.tuwien.ac.at; p.raith@dsg.tuwien.ac.at; a.morichetta@dsg.tuwien.ac.at; dustdar@dsg.tuwien.ac.at).

V. Casamayor Pujol and S. Dustdar are with the Department of Engineering, Universitat Pompeu Fabra, Barcelona, Spain and Dustdar is with ICREA (e-mail: v.casamayor@upf.edu; schahram.dustdar@upf.edu).

able, ranging from well-observed configurations to previously unseen ones. Consequentially, service providers can make decisions about the risk and values of offering a wide range of configurations and compositions.

This requires an uncertainty-aware performance model that can (1) predict system performance and its inherent variability for arbitrary service configurations and compositions, and (2) quantify epistemic uncertainty arising from limited empirical evidence, particularly for unseen configurations. Such uncertainty distinguishes configurations that are reliably known to perform poorly from those whose potential has yet to be established. This information can subsequently be used by optimization and exploration methods to balance performance, SLO risk, and the acquisition of additional evidence.

While the application performance is naturally dependent on the quality provided by the individual services, thewe believe that these two must be evaluated independently from each other Disentangling the service performance from the application performance creates a clear abstraction between the individual services and enables architectures wher

II. Preliminaries

A. Motivating Example

Suppose you own a sensing device, e.g., a smart watch, or a weather station, and you want to build an application that gives you insights into the generated sensor data at real time. This commonly involves multiple processing stages, e.g., to curate data, extract features, or detect pattern, and each of this steps can be configured individually. Because you are interested in

B. Research Challenges & Requirements

Guarantees (RC1) Need a guarantee and not just a target percentile that can err into both directions.

Evidence (RC1) How to give guarantees without e2e empirical evidence.

Arrival Rates (RC2) Requests load varies and requests can arrive at random (worst case), with some jitter (average case), or periodically (best case).

C. Assumptions

Boris ▶*Do we put this here, in the method, or in the implementation?*◀

To ensure independence of composable services, we explicitly decouple host-level resource interaction from workload propagation:

1) *Performance isolation:* Services run in isolated execution environments (e.g., containerized instances with strict CPU and memory allocations), preventing direct resource interference or contention across services.
2) *Workload propagation:* Any upstream service s_1 shapes the arrival process (e.g., request burstiness and throughput) for a downstream service s_2. The performance model of s_2 explicitly includes the parameterized workload characteristics output by s_1. Thus, degradation in s_1's performance naturally propagates through s_2's performance model via its arrival rate λ.
3) *Independent global quality:* We're looking into service compositions where the service quality q of any service $s = \langle r, c, \lambda \rangle$ is determined locally by $q = f_i(c)$, with one global quality parameter $\theta \subseteq c$ across the composition. Hence, the service rate μ only depends on the global quality θ, and not on the quality of any upstream service.
4) *Gaussian-distributed service behavior*: We model the service behavior using Gaussian distributions; this simplifies the PPG composition because we can obtain closed-form Moment Generating Functions (MGFs). While this cannot capture heavy-tail behavior of services—for instance, during major timeouts [4]—it offers a tradeoff between prediction accuracy and compositional efficiency, which is analyzed in Section V-B5.

D. Background and Related Work

1) Analytical Performance Guarantees: Iyer et al. [14] pursue a closely related goal: they aim to provide performance guarantees for software-defined components across configurations that have not been empirically benchmarked. To achieve this, they establish performance contracts for individual components and compose them recursively through algebraic operations. However, two fundamental differences distinguish our work. First, Iyer et al. derive component guarantees by performing code analysis on execution paths. In contrast, microservices are commonly provided as third-party, black-box components [**?**], [**?**], which makes it impractical to inspect their source-code; instead, our presented approach learns performance guarantees purely from runtime observations, thus conserving the black-box character of microservices. Second, while code-level contracts assume deterministic execution bounds suitable for packet-level network functions, distributed application microservices face severe runtime jitter, garbage collection pauses, and resource contention. Our methodology explicitly accounts for this by leveraging Stochastic Network Calculus and Gaussian Process uncertainty to provide probabilistic, queueing-aware end-to-end guarantees.

Franks et al. [9]

Duan [5]

2) Predictive Performance Modeling: Cloud workloads are often a combination of different services and steps [3]; to evaluate and optimize their performance, it is therefore an obvious choice to model the end-to-end performance of workloads. This design—we call it a *top-down* approach—is less complex because is does not model individual services, while assuming homogeneous processing nodes; on the other hand, we argue that it is less flexible—compared to a *bottom-up* approach—because replacing or extending the application's services or allocating more resources to one of them will invalidate the entire end-to-end benchmark. This top-down approach is a common pattern in related work: *Ernest* [21] learn a parametric model that captures the performance of batch analytics jobs in small-scale profiling runs—parameterized by the size of the dataset a and the number of machines k—to then extrapolate performance onto larger deployments; and *CherryPick* [2] gradually learns to model the performance and price of Cloud applications using Gaussian Process regression—a

choice motivated by the cost of empirically evaluating cloud configurations. As reported by Khilji et al. [15], there is also a large gap between learning-based autoscalers developed in research, and everyday production systems often based on simple heuristics. As one example, Google's *Autopilot* [19] avoids predictive modeling entirely, and infers resource limits entirely based on reactive, time-series heuristics and static safety margins.

TODO: Gandhi et el. [11] also learns a performance function to later optimize the system

TODO: Microscaler [23] should be similar to Alipourfard

TODO: text below is still generated and must be confirmed and revised

These approaches reveal a fundamental gap in performance evaluation: they cannot compose individual service models into end-to-end pipeline guarantees, particularly for unseen configurations and combinations. Parametric models like Ernest fail to capture multi-dimensional software configurations (c_s, r_s), black-box GP models like CherryPick require end-to-end re-benchmarking whenever a pipeline topology changes, and operational tools like Autopilot lack queueing-aware delay abstractions. Our work fills this gap by establishing an offline, zero-shot evaluator that translates local Sparse Variational GP predictions into formal Stochastic Network Calculus ($\otimes$) service curves, delivering compositional tail-latency bounds for unobserved microservice configurations.

3) Other references on QoS of Service Compositions: Geebelen et al. [12]
Gallotti et al. [10]
Fanjiang et al. [8]
Wu et al. [22]
Liang et al. [16]

III. Methodology

We define a service pipeline S, where each service $s \in S$ is characterized by $s = \langle r, c, \lambda \rangle$, with r being the allocated resources (e.g., CPU cores), c the service configuration (e.g., DNN model size), and λ the request arrival rate. **Boris** ▶***Actually this should include the service rate μ, which is itself a function of c and r. This definition also ensures that we stay aligned with queuing theory.***◀

Given a pipeline S, our methodology derives a Probabilistic Performance Guarantee (PPG) for S. Such a PPG characterizes the performance of a service or service composition together with its inherent runtime variability and uncertainty resulting from limited empirical evidence. Conceptually, a PPG can be established for individual services (i.e., a single-service pipeline with $|S| = 1$); these local bounds can then be composed recursively to provide end-to-end performance guarantees for multi-service pipelines. Notably, a PPG does not prescribe any optimization objective or algorithm. Instead, they provide fundamental performance guarantees that enable downstream frameworks to optimize for objectives such as latency, throughput, or cost.

Our methodology consists of three main steps, as depicted in Figure 1: (1) learning individual service's probabilistic performance model from empirical observations, (2) deriving statistically valid service-level bounds from the model, and (3) composing these bounds into a PPG for the complete service pipeline.

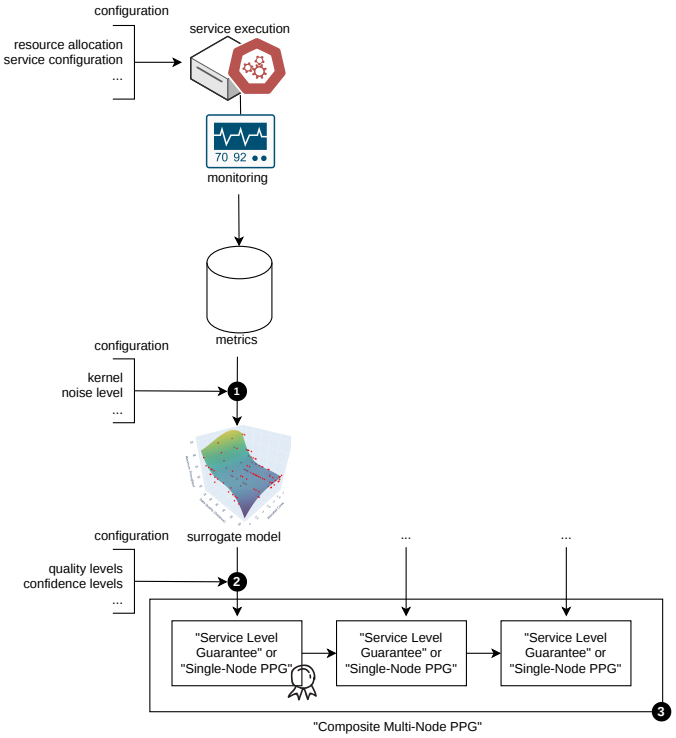


Fig. 1: High-level overview of our three-step methodology for creating end-to-end performance guarantees of arbitrary service composition and configurations: ❶ we model service behavior through a surrogate function, ❷ we extract probabilistic performance guarantees (PPGs) around individual services, and ❸ we calculate the expected performance levels across service compositions by concatenating their PPGs.

A. Learning

The first step trains a probabilistic model that predicts the performance of an individual service configuration alongside its associated epistemic uncertainty. Two constraints that apply to this step are: (1) scaling behavior of services is rarely linear [2], e.g., performance saturates after providing sufficient RAM, while increasing parallelism can actually result in degraded performance due to scheduling overhead; also (2) services live in containers, whose source code can rarely be analyzed to make assumptions about their functional behavior. To satisfy these constraints, Gaussian Processes (GPs) are a fitting solution as they are non-parametric and do not impose any structure of the functions learned.

To maintain computational tractability as operational performance data accumulates over time, we employ scalable Gaussian Process (GP) approximations, such as inducing-point methods (e.g., Sparse Variational GPs), to bound inference complexity.

By leveraging the Gaussian predictive distribution of the SVGP, the Moment Generating Function (MGF) of unit ex-

ecution time $M_Y(\theta) = \mathbb{E}[e^{\theta Y}] = \exp(\theta\mu_* + \frac{\theta^2}{2}\mathbb{V}[Y])$ exists in exact closed form. This allows us to convert the empirical execution time of the black-box worker into a closed-form service curve bound $S(t)$ within our $(\min, +)$ Stochastic Network Calculus (SNC) engine.

B. Service Bounds

The second step transforms the uncertainty output of the performance model into a statistically valid service-level performance bound. This derivation relies on three standard modeling assumptions: (1) the appropriateness of the chosen surrogate model and kernel for capturing the underlying system behavior, (2) the adequacy of the observation noise model in representing aleatoric runtime variability, and (3) the statistical representativeness of the training observations relative to operational conditions.

The second step transforms the SVGP predictive distribution into a statistically valid service-level bound. A key property of standard Gaussian Process surrogates is their assumption of Gaussian-distributed observational noise, which possesses light, exponentially decaying tails. While real-world microservice tail spikes can occasionally exhibit heavy-tailed behavior (e.g., during major garbage collection pauses), the Gaussian assumption provides a critical analytical benefit: it guarantees the existence of closed-form MGFs. This enables exact, low-latency $(\min, +)$ algebraic composition $(\otimes)$ across multi-stage pipelines without resorting to costly numerical Monte Carlo convolutions. For operational SLA verification targeting standard percentiles (P_{90}–P_{99}), this formulation offers an optimal balance between prediction accuracy and compositional efficiency.

C. Composition & Inter-Service Dependencies

The third step composes individual service-level bounds according to the topology of S to derive the PPG for the overall service pipeline. For sequential topologies, composition is achieved by leveraging the structural properties of Stochastic Network Calculus (SNC), explicitly accounting for individual violation probabilities. Ideally, we can use the native min-plus convolutions in SNC for concatenating PPGs.

TODO: To justify the synergy and technology choice for each step in the methodology we can use something like the following.

Evaluating microservice pipelines under unseen configurations demands a modeling framework that balances nonlinear expressiveness with rigorous uncertainty quantification. Empirical machine learning techniques (e.g., Neural Networks, Gradient Boosted Trees) yield deterministic point estimates that lack epistemic uncertainty, making them prone to silent over-confidence under data sparsity and incompatible with Stochastic Network Calculus (SNC). Frequentist parametric approaches (e.g., Ernest) rely on rigid scaling equations that fail to capture complex multi-dimensional configuration spaces (c_s, r_s).

In contrast, our framework employs a Bayesian non-parametric surrogate, specifically Sparse Variational Gaussian Processes (SVGPs). The Bayesian approach offers three key advantages: (1) it gracefully handles sparse offline profiling through prior regularization; (2) it explicitly decomposes total variance into aleatoric system jitter and epistemic model ignorance; and (3) its Gaussian predictive distribution translates directly into closed-form Moment-Generating Functions (MGFs). This mathematical synergy enables exact, sub-millisecond $(\min, +)$ queueing composition $(\otimes)$ across multi-stage pipelines without sacrificing tail-bound guarantees.

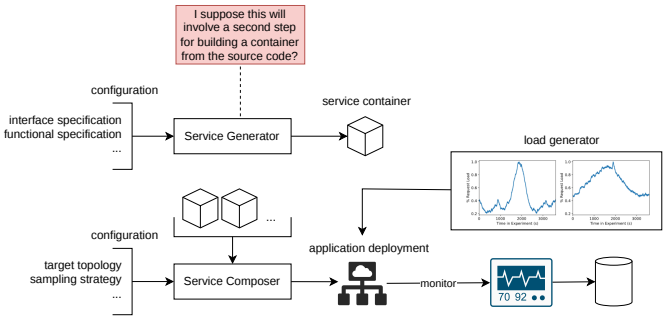


Fig. 2: High-level architecture of the experimental environment: the *service generator* creates containerized services according to a desired interface and functionality; the *service composer* ...

IV. Implementation

Discretization of the Configuration Space. Numerous service knobs could actually be scaled on a fine-grained Integer scale, for example, the scheduled CPU quota is configured in *microseconds*, or the resolution of video frames in *pixel*. Nevertheless, Cloud systems commonly offer only a small set of discretized configurations, around which the performance predictions are more accurate [2]. We hypothesize that the granularity of the discretization might improve the convergence of accurate PPGs, while trading off solution optimality. Not sure if we'd analyze this now.

V. Evaluation

A. Experimental Setup

Service Generation. To evaluate generalization across structural dimensions, we evaluate unseen service configurations, unseen service compositions, and their combination. This requires an ample set of services and service compositions; for this, we combine an existing set of microservices and configuration interfaces from prior work [20] and augment it through LLM-based code generation.

Sampling Strategies. To evaluate the impact of training data collection strategies on bound calibration, empirical observations will be gathered under three *sampling strategies*: (1) uniform random sampling across the configuration space, (2) active learning under different acquisition functions (e.g., uncertainty minimization of value maximization), and (3) exhaustive grid search, which we must prune at some point.

Baselines. For each evaluation setting, empirical end-to-end measurements collected from deployed testbed services serve as ground truth against which inferred bounds and PPGs

are benchmarked. We conduct comprehensive component-wise ablations and baseline comparisons across three stages:

- **Prediction Models:** Gaussian Processes (GPs), Empirical ML methods including Random Forests, XGBoost, and (Deep) Neural Networks, together with standard/ridge regression.
- **Uncertainty Quantification & Calibration:** GP posterior variance intervals, quantile regression, conformal prediction (including α-budgeting to support composition), and ensemble-based variance estimation
- **Compositional Operators:** standard SNC incl. Moment Generating Function (MGF); non-MGF SNC (only makes sense for distributions with heavy tails—hence, the model **must be** non Gaussian), which could mean some sort of Monte Carlo simulation; naive composition omitting uncertainty; and traditional queuing networks.

B. Coverage and Tightness of Service Bounds

1) Base Case: This will use most of the baselines from above to evaluate the maturity of our ideas vs. existing techniques.

2) Varying Evidence per Service: This makes use of the presented sampling strategies. The services are executed somewhere under an arbitrary service composition that is different from the target composition. We gradually train the PPGs with more of this data and see if this has an impact on the quality of the unseen target composition.

3) Varying Chain Depth: We evaluate how the validity, empirical coverage rate, and tightness of the derived PPGs scale with pipeline depth. By evaluating synthetic and real chains across varying lengths $|S| \in \{1, 2, 5, 10, 20, 50, 100\}$, we assess whether the composed bounds remain sufficiently tight and actionable for downstream optimization without suffering from severe bound degradation.

4) Varying Load and Arrival Pattern: Here we will use different arrival pattern (e.g., periodic, random (= Poisson), and batching) and different arrival rates (e.g., diurnal or bursty with custom range). I already explored this once and found that we can characterize each of these arrival processes through the squared coefficient of variation (C_a^2) as defined by

$$C_a^2 = \frac{\mathrm{Var}(X)}{(\mathbb{E}[X])^2} = \frac{\sigma^2}{\mu^2} \tag{1}$$

5) Varying Tail Behaviors: Here I can look at services that have a different tail behavior (e.g., increasingly heavy/light tails) and how this impacts the quality of the guarantee.

6) Varying Percentiles: Optionally, I could also look into different percentile guarantees and show where each method is more/less accurate.

C. Iterative Optimization of Service Bounds

I would integrate this into a use case, where we have to compose increasingly better applications using this method.

1) Fine-Tuning Bounds with E2E Samples: Our focus is giving a rough guarantee BEFORE deploying it e2e, so this means the best we can do as soon as it goes e2e is to show that our model provides an initial accuracy boost that likely vanishes over time.

2) Inferring Faster Service Compositions: This can go into two directions, first, we can optimize the service configuration according to the model, and second, we can improve the service composition.

For the first one, we can take *Fluxion* [16] (cf. Section 5.1.3) as inspiration, where they also used the model paired with an external optimizer to show how their model enables better performance optimization. They simply compared the different guarantee models (theirs, a plain GP, and a DNN) for coupled with different optimizers; this we can do as well, and it is also a nice cut between the guarantee (our contribution) and the external optimizer.

For the second one, it might be a bit more complex because we have not talked a lot about the composition mechanism, but I would design it like the first one: we assume some composition mechanism that is external to us and we gradually show that by using our guarantees it can create better service composition. I think that this is the real deal because it repeatedly looks into unseen configurations, but it also the most complex evaluation. In any case, I think that a genetic algorithm could be the best choice here, because it builds a population of potential solutions and then iteratively improves them through sampling. But we do NOT have to include multiple rounds of benchmarking, but only one. The distinction would be:

- **single round**: we build a population of compositions and grade them according to different guarantees (ours vs. baselines). We repeat this step in a genetic algorithm until we have a solution for every guarantee. Then we benchmark these solutions and look at their fitness.
- **multiple round**: as above, but we use the empirical samples we captured from the benchmark and run the genetic algorithm (planning) and benchmarking again and again until we converge to a satisfying result. This we keep for future work at NSDI hehe

VI. Discussion

@Víctor most likely

VII. Conclusion

@Víctor most likely

References

[1] Aiello, M.: A Paradigm Shift in Service Research: The Case of Service Composition. IEEE Transactions on Services Computing **18**(3), 1213–1215 (May 2025). doi:10.1109/TSC.2025.3552345

[2] Alipourfard, O., Liu, H.H., Chen, J., Venkataraman, S., Yu, M., Zhang, M.: CherryPick: Adaptively Unearthing the Best Cloud Configurations for Big Data Analytics. In: 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17). pp. 469–482 (2017)

[3] Buyya, R., Yeo, C.S., Venugopal, S., Broberg, J., Brandic, I.: Cloud computing and emerging IT platforms: Vision, hype, and reality for delivering computing as the 5th utility. Future Generation Computer Systems **25**(6), 599–616 (Jun 2009). doi:10.1016/j.future.2008.12.001

[4] Dean, J., Barroso, L.A.: The tail at scale. Communications of the ACM **56**(2), 74–80 (Feb 2013). doi:10.1145/2408776.2408794

[5] Duan, Q.: Modeling and performance analysis for composite network–compute service provisioning in software-defined cloud environments. Digital Communications and Networks **1**(3), 181–190 (Aug 2015). doi:10.1016/j.dcan.2015.05.003

[6] Esashi, A., Lertpongrujikorn, P., Kato, S., Salehi, M.A.: Action Engine: Automatic Workflow Generation in FaaS. Future Generation Computer Systems **174**, 107947 (Jan 2026). doi:10.1016/j.future.2025.107947

[7] Fan, A., Gokkaya, B., Harman, M., Lyubarskiy, M., Sengupta, S., Yoo, S., Zhang, J.M.: Large Language Models for Software Engineering: Survey and Open Problems. In: 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE). pp. 31–53 (May 2023). doi:10.1109/ICSE-FoSE59343.2023.00008

[8] Fanjiang, Y.Y., Syu, Y., Kuo, J.Y.: Search based approach to forecasting QoS attributes of web services using genetic programming. Information and Software Technology **80**, 158–174 (Dec 2016). doi:10.1016/j.infsof.2016.08.009, https://www.sciencedirect.com/science/article/pii/S0950584916301409

[9] Franks, G., Al-Omari, T., Woodside, M., Das, O., Derisavi, S.: Enhanced Modeling and Solution of Layered Queueing Networks. IEEE Transactions on Software Engineering **35**(2), 148–161 (Mar 2009). doi:10.1109/TSE.2008.74

[10] Gallotti, S., Ghezzi, C., Mirandola, R., Tamburrelli, G.: Quality Prediction of Service Compositions through Probabilistic Model Checking. In: Becker, S., Plasil, F., Reussner, R. (eds.) Quality of Software Architectures. Models and Architectures. pp. 119–134. Springer, Berlin, Heidelberg (2008). doi:10.1007/978-3-540-87879-7_8

[11] Gandhi, A., Harchol-Balter, M., Das, R., Lefurgy, C.: Optimal power allocation in server farms. In: Proceedings of the eleventh international joint conference on Measurement and modeling of computer systems. pp. 157–168. SIGMETRICS '09, Association for Computing Machinery, New York, NY, USA (Jun 2009). doi:10.1145/1555349.1555368

[12] Geebelen, D., Geebelen, K., Truyen, E., Michiels, S., Suykens, J.A.K., Vandewalle, J., Joosen, W.: QoS prediction for web service compositions using kernel-based quantile estimation with online adaptation of the constant offset. Information Sciences **268**, 397–424 (Jun 2014). doi:10.1016/j.ins.2013.12.063

[13] Guo, Y., Wu, M., Cao, Y., Wei, J., Ran, D., Yang, W., Xie, T.: BackendForge: Benchmarking Agentic End-to-End Code Generation with Backend Services (Jul 2026). doi:10.48550/arXiv.2607.11042, arXiv:2607.11042 [cs.SE]

[14] Iyer, R., Pedrosa, L., Zaostrovnykh, A., Pirelli, S., Argyraki, K., Candea, G.: Performance Contracts for Software Network Functions. In: 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19). pp. 517–530. USENIX Association (Feb 2019)

[15] Khilji, S.I., Furutanpey, A., Dustdar, S.: Incentives and Evidence in Learned Service Orchestration (Jun 2026). doi:10.48550/arXiv.2606.16555, arXiv:2606.16555 [cs.DC]

[16] Liang, C.J.M., Fang, Z., Xie, Y., Yang, F., Li, Z.L., Zhang, L.L., Yang, M., Zhou, L.: On Modular Learning of Distributed Systems for Predicting End-to-End Latency. pp. 1081–1095 (2023), https://www.usenix.org/conference/nsdi23/presentation/liang-chieh-jan

[17] Meflah, W., Brabra, H., Acheli, M., Douma, H.B., Sellami, M., Gaaloul, W., Zeghlache, D.: From Natural Language to TOSCA: Leveraging LLMs for Automated Service Composition. In: Aiello, M., Deng, S., Murillo, J.M., Georgievski, I., Benatallah, B., Wang, Z. (eds.) Service-Oriented Computing. pp. 3–18. Springer Nature, Singapore (2026). doi:10.1007/978-981-95-5012-8_1

[18] Papazoglou, M.P., Georgakopoulos, D.: Introduction: Service-oriented computing. Communications of the ACM **46**(10), 24–28 (Oct 2003). doi:10.1145/944217.944233

[19] Rzadca, K., Findeisen, P., Swiderski, J., Zych, P., Broniek, P., Kusmierek, J., Nowak, P., Strack, B., Witusowski, P., Hand, S., Wilkes, J.: Autopilot: workload autoscaling at Google. In: Proceedings of the Fifteenth European Conference on Computer Systems. pp. 1–16. EuroSys '20, New York, NY, USA (Apr 2020). doi:10.1145/3342195.3387524

[20] Sedlak, B., Raith, P., Morichetta, A., Pujol, V.C., Dustdar, S.: Multi-Dimensional Autoscaling of Stream Processing Services on Edge Devices. IEEE Transactions on Services Computing pp. 1–13 (2026). doi:10.1109/TSC.2026.3711142

[21] Venkataraman, S., Yang, Z., Franklin, M., Recht, B., Stoica, I.: Ernest: Efficient Performance Prediction for Large-Scale Advanced Analytics. In: 13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16). pp. 363–378 (2016)

[22] Wu, Q., Zhang, M., Zheng, R., Lou, Y., Wei, W.: A QoS-Satisfied Prediction Model for Cloud-Service Composition Based on a Hidden Markov Model. Mathematical Problems in Engineering **2013**(1), 387083 (2013). doi:10.1155/2013/387083, https://onlinelibrary.wiley.com/doi/abs/10.1155/2013/387083

[23] Yu, G., Chen, P., Zheng, Z.: Microscaler: Cost-Effective Scaling for Microservice Applications in the Cloud With an Online Learning Approach. IEEE Transactions on Cloud Computing **10**(2), 1100–1116 (Apr 2022). doi:10.1109/TCC.2020.2985352

POTENTIAL APPENDIX

- Proof why we need to model latency directly and not capacity. Plot the noise and the shapiro-wilk normality test.
- Proof of the dependence/independence between frames services in the queue.
- List of the services and their states and configuration knobs in more detail. This can be clusters of service types, as for *Fluxion* in [16].